

CMPE 492

VeriFi: A Reputation-Based Platform for  
Verifiable Financial Predictions

Arda Saygan  
Abdurrahman Arslan  
Çetin Tiryaki  
Volkan Bora Seki  
Yusuf Anıl Yazıcı

Advisor:  
Hüseyin Birkan Yılmaz

# Contents

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>INTRODUCTION</b>                            | <b>1</b>  |
| 1.1      | Broad Impact . . . . .                         | 1         |
| 1.2      | Ethical Considerations . . . . .               | 2         |
| <b>2</b> | <b>PROJECT DEFINITION AND PLANNING</b>         | <b>4</b>  |
| 2.1      | Project Definition . . . . .                   | 4         |
| 2.2      | Project Planning . . . . .                     | 5         |
| 2.2.1    | Project Time and Resource Estimation . . . . . | 5         |
| 2.2.2    | Success Criteria . . . . .                     | 7         |
| 2.2.3    | Risk Analysis . . . . .                        | 7         |
| 2.2.4    | Team Work . . . . .                            | 7         |
| <b>3</b> | <b>RELATED WORK</b>                            | <b>10</b> |
| <b>4</b> | <b>METHODOLOGY</b>                             | <b>12</b> |
| <b>5</b> | <b>REQUIREMENTS SPECIFICATION</b>              | <b>14</b> |
| 5.1      | Stakeholders and User Personas . . . . .       | 14        |
| 5.2      | Functional Requirements . . . . .              | 14        |
| 5.2.1    | Authentication . . . . .                       | 14        |
| 5.2.2    | Hard Claim Creation . . . . .                  | 16        |
| 5.2.3    | LLM Claim Extraction (Planned) . . . . .       | 16        |
| 5.2.4    | Claim Resolution . . . . .                     | 16        |
| 5.2.5    | Cryptographic Proofs . . . . .                 | 16        |
| 5.2.6    | Truth Score . . . . .                          | 16        |
| 5.2.7    | Feed . . . . .                                 | 19        |
| 5.2.8    | User Profiles . . . . .                        | 20        |
| 5.3      | Non-Functional Requirements . . . . .          | 20        |
| 5.3.1    | Security . . . . .                             | 20        |
| 5.3.2    | Privacy and GDPR / KVKK . . . . .              | 20        |
| 5.3.3    | Performance and Reliability . . . . .          | 20        |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>6</b> | <b>DESIGN</b>                              | <b>23</b> |
| 6.1      | Information Structure . . . . .            | 23        |
| 6.2      | Information Flow . . . . .                 | 25        |
| 6.3      | System Design . . . . .                    | 28        |
| 6.4      | Authentication Design . . . . .            | 29        |
| 6.5      | User Interface Design . . . . .            | 31        |
| 6.6      | Design Decisions and Constraints . . . . . | 31        |
| <b>7</b> | <b>IMPLEMENTATION AND TESTING</b>          | <b>33</b> |
| 7.1      | Implementation . . . . .                   | 33        |
| 7.2      | Testing . . . . .                          | 34        |
| 7.3      | Deployment . . . . .                       | 35        |
| <b>8</b> | <b>RESULTS</b>                             | <b>36</b> |
| <b>9</b> | <b>CONCLUSION</b>                          | <b>37</b> |

# Chapter 1

## INTRODUCTION

### 1.1 Broad Impact

The financial media landscape is plagued by unverifiable claims. Influencers, analysts, and self-proclaimed experts routinely make bold financial predictions on social media with no accountability mechanism. When predictions prove correct they are amplified; when wrong, they are quietly deleted or ignored. This asymmetry erodes public trust and enables misinformation to thrive in financial discourse.

Consider a concrete example. A popular financial influencer on Twitter (X) posts: “Bitcoin will crash 50% by next Friday — sell everything now.” Thousands of followers see the post and some act on it. When Bitcoin instead rises sharply, the influencer quietly deletes the tweet and later claims he never made such a prediction. A retail investor who lost money following the advice has no recourse: the tweet is gone, there is no proof it ever existed, and the influencer’s public track record remains unblemished. On existing platforms this pattern repeats daily, because there is no technical mechanism to hold predictors accountable after deletion. The influencer’s reputation is built entirely on selectively showcasing correct calls while erasing incorrect ones — a survivorship bias that misleads followers at scale.

Now imagine the same scenario on VeriFi. When the influencer posts his prediction, the platform separates the social commentary from the structured financial claim — “Bitcoin, Bearish, 50%, by next Friday” — and cryptographically signs the claim with the influencer’s private key. A skeptical follower clicks *Save Proof* and downloads the signed claim to their local device. When the prediction fails and the influencer deletes the post, the social text disappears (respecting the right to erasure), but the mathematical proof persists. The follower uploads the saved proof to VeriFi’s public verification page, the

system confirms the cryptographic signature matches the influencer’s wallet, and the failed prediction is permanently recorded in the influencer’s Truth Score. Deletion does not equal deniability.

VeriFi addresses this problem by building a social media platform where every financial prediction is cryptographically signed at the time of posting. Because the signature is generated from the author’s private key and bound to the claim content, a prediction cannot be retroactively modified or plausibly denied — even after the original post is deleted. This property, *non-repudiation*, is the core differentiator of VeriFi compared to existing financial social platforms.

The platform benefits several stakeholder groups. Retail investors gain access to a transparent, tamper-proof track record of financial influencers, enabling more informed decisions about whose predictions to follow. Skilled predictors accumulate a credible, verifiable reputation that serves as a professional credential. Researchers and journalists studying financial misinformation gain a public dataset of signed, timestamped predictions with resolution outcomes.

At a societal level, VeriFi promotes financial literacy by shifting discourse toward verifiable, quantified claims and away from vague, unfalsifiable commentary. By making prediction accuracy publicly auditable, the platform creates an economic incentive for quality analysis and a natural disincentive for low-effort, clickbait content.

## 1.2 Ethical Considerations

Several ethical dimensions require careful attention in the design and operation of VeriFi.

**Not investment advice.** VeriFi is a reputation-tracking platform, not a financial advisory service. All content is user-generated and not endorsed by the platform operators. The platform displays a prominent disclaimer that no content constitutes investment advice, in compliance with applicable financial regulations.

**Privacy and data protection.** VeriFi deliberately avoids collecting personally identifiable information (PII). Users are identified solely by their Ethereum wallet address, a pseudonymous identifier derived from a cryptographic key pair. No names, email addresses, phone numbers, or other PII are stored. Post text is deletable by the user to accommodate GDPR and KVKK (Turkish Personal Data Protection Law) rights to erasure. However, the structured claim payload — the financially verifiable component of a prediction — persists even after post deletion. This design reflects the principle

that while personal commentary may be retracted, quantified public financial claims carry a different epistemic status analogous to a published prediction.

**AI limitations.** The planned LLM-based claim extraction feature assists users in structuring natural-language posts into verifiable claims. The system is designed so that the LLM serves as an assistant, not a decision-maker: every extracted claim is presented to the user for explicit confirmation, editing, or rejection before publication. The LLM never publishes claims autonomously. The system also flags when a prediction lacks a measurable target or timeframe, prompting the user to provide those fields rather than inferring them, in order to maintain verifiability.

**Sybil resistance.** A reputation system is only as trustworthy as its identity layer. By anchoring identity to a cryptographic key pair rather than a platform-controlled account, VeriFi ensures that identity manipulation requires actual cryptographic key control. The challenge-response authentication scheme prevents replay attacks. Planned features such as stake-weighted voting for soft claims further raise the economic cost of Sybil attacks.

**Fairness in resolution.** Claim resolution in v1 is performed by administrators identified by a whitelisted Ethereum address. Automated oracle-based resolution is planned for future milestones. The platform is designed so that resolved claim status cannot revert, and all resolution actions are tied to the admin’s cryptographic identity, making resolution decisions auditable.

## Chapter 2

# PROJECT DEFINITION AND PLANNING

### 2.1 Project Definition

VeriFi is a web-based social media platform for verifiable financial predictions. The central concept is the *Hard Claim*: a quantified, time-bounded financial prediction with the structured fields `{asset, direction, percentage, until}`. Hard claims are cryptographically signed by their author at publication time. When the deadline passes, each claim is resolved as confirmed or rejected by comparing the actual asset price movement against the prediction, either automatically via a financial data oracle or manually by an administrator.

Each user accumulates a *Truth Score*, a global reputation metric derived from the accuracy and difficulty of their resolved claims. The Truth Score is publicly visible, non-editable, and tamper-evident because it is anchored to cryptographically signed claim records.

Authentication is passwordless and wallet-based. Users either generate a native VeriFi wallet (12-word BIP39 mnemonic [3], secp256k1 key derivation via BIP32 [8], private key stored encrypted in browser `localStorage`) or connect an existing MetaMask wallet. Login uses an EIP-191 `personal_sign` challenge-response flow [5]: the server issues a single-use nonce, the client signs it with the private key, and the server recovers the signing address for verification. No password is ever stored or transmitted.

The platform is built as a monorepo with a Django REST Framework backend (deployed on Render) and a Vite/React/TypeScript frontend (deployed on Vercel). The database is SQLite in development and PostgreSQL in production.

**In scope (v1):**

- Wallet-based authentication (native BIP39 wallet and MetaMask)
- Hard claim creation, feed, and user profiles
- Claim lifecycle management (`undetermined` → `confirmed` / `rejected`)
- Manual claim resolution by admin
- Cryptographic proof generation and download
- Standalone proof verification page (no login required)
- Truth Score computation and public display

**Out of scope (v1):**

- Soft claims and community voting
- Automated oracle-based claim resolution (planned Milestone 3)
- LLM-based automatic claim extraction (stub exists; full feature in progress)
- Domain-specific reputation scores
- Monetization and subscription tiers
- Mobile applications
- Blockchain anchoring / on-chain timestamping

## 2.2 Project Planning

### 2.2.1 Project Time and Resource Estimation

The project follows a milestone-driven development schedule aligned with the course calendar. Four milestones and a final senior presentation have been defined.

The team consists of five members, each contributing across frontend and backend work. External resource dependencies include financial data APIs (CoinGecko, Yahoo Finance, Alpha Vantage) for oracle-based resolution, and a locally hosted language model (via LM Studio) for LLM claim extraction.



Table 2.1: Project milestones and their key deliverables.

| <b>Milestone</b> | <b>Date</b>   | <b>Deliverables</b>                                                                                                                               |
|------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| M1               | March 8, 2026 | Authentication and identity: native wallet (BIP39/secp256k1), MetaMask integration, challenge-response JWT login, unified login UI                |
| M2               | April 5, 2026 | Core social and claim pipeline: post composer, LLM claim extraction (Agno), claim card UI, signed proof packages, public social feed              |
| M3               | May 17, 2026  | Resolution, reputation and verification: oracle integration (CoinGecko, Yahoo Finance), Truth Score, proof verifier page, feed filtering by asset |
| Final            | June 11, 2026 | End-to-end demo, UI/UX polish, edge case handling, performance pass, final documentation                                                          |

### **2.2.2 Success Criteria**

The project will be considered successful if the following criteria are met by the final presentation:

- A registered user can create a hard claim; the claim is cryptographically signed and stored.
- A visitor can browse the public claim feed without registering.
- An expired claim can be resolved (confirmed or rejected) and the author's Truth Score is updated accordingly.
- Any third party can download a proof file and verify it on the standalone proof verification page without logging in.
- The system correctly rejects tampered or forged proof files.
- The feed page loads within 2 seconds under typical usage conditions.
- No critical security vulnerabilities exist in the authentication or claim submission flow.

### **2.2.3 Risk Analysis**

### **2.2.4 Team Work**

The project team consists of five members. Responsibilities are distributed as follows:

The team holds weekly advisory meetings on Tuesdays (17:30–18:00) with the project advisor. Meeting notes and decisions are recorded in the project GitHub wiki. Rush weekends with concentrated development effort have been scheduled around milestone deadlines.

Table 2.2: Identified risks, likelihood, impact, and mitigation strategies.

| <b>Risk</b>                                        | <b>Likelihood</b> | <b>Impact</b> | <b>Mitigation</b>                                                                                            |
|----------------------------------------------------|-------------------|---------------|--------------------------------------------------------------------------------------------------------------|
| Oracle API down-time or rate limiting              | Medium            | High          | Exponential backoff retries; fallback to secondary data source; manual admin resolution as last resort       |
| LLM extraction quality insufficient                | High              | Medium        | v1 ships with manual structured entry form; LLM extraction is an enhancement, not a blocker                  |
| Truth Score formula disputes                       | Medium            | Medium        | Formula publicly documented and fixed at v1 launch; transparent claim records allow independent verification |
| Browser <code>localStorage</code> key loss         | Medium            | High          | Mnemonic displayed and backup strongly encouraged at registration; no server-side key recovery by design     |
| Private key leakage from <code>localStorage</code> | Low               | Critical      | AES-256-GCM encryption with PBKDF2-SHA256 (100,000 iterations); key never transmitted to server              |
| GDPR / KVKK compliance                             | Low               | High          | No PII collected; post text deletable; claim payloads treated as public reputation data                      |

Table 2.3: Team member responsibilities.

| Member             | Primary Responsibilities                                                                            |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Arda Saygan        | Backend architecture; hard claim data models and API endpoints; project repository management       |
| Abdurrahman Arslan | MetaMask integration; frontend PostCard and claim card UI components; requirements specification    |
| Çetin Tiryaki      | LLM experimentation and benchmarking; claim extraction UI; report contributions                     |
| Volkan Bora Seki   | AI/LLM infrastructure (Agno + LM Studio); multilingual claim extraction; LLM benchmark design       |
| Yusuf Anıl Yazıcı  | Native wallet and key management; cryptographic authentication frontend; midterm report; deployment |

## Chapter 3

# RELATED WORK

The problem of tracking and verifying financial predictions has been approached from several directions in both industry and academia.

**Prediction Markets.** Prediction markets are exchange-traded platforms where users buy and sell contracts on future events, with prices reflecting aggregate probability estimates [7]. Platforms such as Polymarket and PredictIt allow users to stake real money on event outcomes, providing strong accountability: incorrect predictions result in direct financial loss. While effective for accountability, these systems are regulated as financial instruments in many jurisdictions and do not support the social media dynamics — posts, feeds, followers, profiles — that VeriFi targets. Augur [4] extends the model to a decentralised, blockchain-based architecture, but requires on-chain interaction and cryptocurrency holdings, creating a high barrier for casual users.

**Social Trading Platforms.** eToro and similar platforms allow users to publish and copy investment portfolios, providing a form of reputation tracking based on actual trade performance. However, these platforms require linking real brokerage accounts, making them inaccessible to users who wish to share predictions without executing trades. StockTwits provides a Twitter-like social layer for financial commentary but lacks any systematic verification or scoring mechanism: predictions and outcomes are not tracked.

**Reputation Systems.** Reputation systems in online communities, such as Stack Overflow’s karma system or academic citation indices, have been studied extensively [2]. A key finding is that reputation scores are only as trustworthy as the underlying identity system and the non-repudiability of the actions they measure. VeriFi applies this insight to financial predictions by anchoring reputation to cryptographic identity and signed claim records.

**Cryptographic Non-Repudiation.** The use of digital signatures for non-repudiation of electronic documents is well established [1]. Ethereum’s EIP-191 `personal_sign` standard [5] extends this to wallet-based signatures,

which VeriFi uses as the foundation of its proof system. The key contribution of VeriFi is applying this mechanism to social media content — specifically financial predictions — so that deletion or denial of a claim becomes cryptographically impossible for anyone who has saved the signed proof.

**LLM-Assisted Structured Extraction.** Recent work on large language model-based information extraction has demonstrated strong performance on converting natural language into structured records [6]. VeriFi uses an LLM (via the Agno framework with a locally hosted model) to assist users in extracting structured `{asset, direction, percentage, timeframe}` tuples from free-form text. A critical design constraint, derived from team discussions, is that the LLM must not infer missing fields — especially timeframes — but must prompt the user to supply them explicitly, preserving the verifiability of every published claim.

# Chapter 4

## METHODOLOGY

VeriFi is built as a monorepo web application with clearly separated frontend and backend layers.

**Backend.** The backend is a REST API built with Django 6 and Django REST Framework (DRF). Authentication uses `djangorestframework-simplejwt` for JWT issuance and validation, and the `eth-account` library for Ethereum signature verification. The backend is stateless: no server-side sessions are maintained. All authentication state is encoded in the JWT token. The database is SQLite in development and PostgreSQL in production (Render).

**Frontend.** The frontend is a React 19 + TypeScript single-page application (SPA) built with Vite 7. UI components use Tailwind CSS v4 and the `shadcn/ui` component library. Cryptographic operations (BIP39 mnemonic generation, BIP32 key derivation, secp256k1 signing, AES-256-GCM key encryption) are performed entirely client-side using `@scure/bip39`, `@scure/bip32`, `@noble/secp256k1`, and `@noble/hashes` — pure TypeScript implementations with no native dependencies. The frontend is deployed on Vercel.

**Authentication.** The authentication system is passwordless and wallet-based. Two wallet modes are supported: (1) a native VeriFi wallet based on BIP39 mnemonic generation and BIP32/BIP44 key derivation, with the private key stored AES-256-GCM encrypted in browser `localStorage`; and (2) MetaMask via the EIP-1193 browser extension API. Both modes use an identical EIP-191 `personal_sign` challenge-response flow for login, described in full in Section 6.4.

**Claim Signing.** Every hard claim is cryptographically signed with the author's secp256k1 private key at creation time. The signature covers a canonical JSON serialisation of the claim payload: `{author_address, asset, direction, percentage, until, server_timestamp}`. This pro-

duces a proof bundle that allows any third party to verify — using only the standard Ethereum public-key infrastructure — that a specific user made a specific prediction at a specific time.

**LLM Claim Extraction.** The system includes a claim extraction endpoint (POST /api/posts/extract-claims/) which, in the planned implementation, will pass post content to an LLM (Agno framework with a locally hosted language model) and return a structured list of detected claims. The LLM output schema is {claims: [{asset, direction, target\_value, target\_unit, timeframe\_iso, language}]}. Each extracted claim is presented to the user as a Claim Card for explicit confirmation, editing, or rejection before publication. The LLM was benchmarked on a custom dataset of English and Turkish prediction sentences spanning single-claim, multi-claim, and edge-case inputs. As of the midterm, the endpoint returns an empty list (stub); the full implementation is in progress.

**Oracle-Based Resolution.** Claim resolution will be automated by a scheduled backend job that, upon claim expiry, queries a financial data oracle (CoinGecko for cryptocurrency assets; Yahoo Finance and Alpha Vantage for stocks, forex, and commodities) to determine whether the price movement matched the prediction. The resolution criterion is: did the asset move at least `percentage%` in the predicted `direction` by the `until` date? Resolution status cannot revert once set, and the resolution is idempotent. This feature is targeted for Milestone 3.

**Truth Score.** Each user’s Truth Score is a single global numeric reputation metric updated automatically on every claim resolution. The formula is designed to weight claim difficulty: harder predictions (wider percentage targets, longer timeframes, more volatile assets) yield greater reputation gains when confirmed and greater penalties when rejected. The exact weighted formula is under active design as an open question (OQ-01 in the project SRS).



# Chapter 5

## REQUIREMENTS SPECIFICATION

### 5.1 Stakeholders and User Personas

Table 5.1: Stakeholders and their primary interactions with the system.

| Persona        | Description                                | Primary Actions                                  |
|----------------|--------------------------------------------|--------------------------------------------------|
| Predictor      | Registered user who makes financial claims | Register, login, create claims, view profile     |
| Visitor        | Unauthenticated user browsing the platform | View feed, view profiles, verify proofs          |
| Admin          | Whitelisted wallet address (Ethereum)      | Manually resolve claim status via API            |
| Proof Verifier | Anyone with a downloaded proof file        | Upload proof to the standalone verification page |

### 5.2 Functional Requirements

#### 5.2.1 Authentication

The system provides passwordless, wallet-based login. No email or password is stored on any server. Two flows are supported: native BIP39 wallet and MetaMask.

Table 5.2: Authentication functional requirements.

| ID      | Requirement                                                                                                                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AUTH-01 | The system <b>MUST</b> generate a 12-word BIP39 mnemonic in-browser on the client without server involvement.                                                                           |
| AUTH-02 | The system <b>MUST</b> derive a secp256k1 key pair from the mnemonic using BIP32 path <code>m/44'/60'/0'/0/0</code> .                                                                   |
| AUTH-03 | The Ethereum address <b>MUST</b> be derived as the last 20 bytes of <code>keccak256(uncompressedPublicKey[1:])</code> , prefixed with <code>0x</code> .                                 |
| AUTH-04 | The user <b>MUST</b> explicitly acknowledge that they have saved their mnemonic before registration proceeds.                                                                           |
| AUTH-05 | The private key <b>MUST</b> be encrypted with AES-256-GCM using a PBKDF2-SHA256 derived key (100,000 iterations, random salt and IV) before being stored in <code>localStorage</code> . |
| AUTH-06 | The server <b>MUST</b> reject duplicate wallet address registrations with HTTP 409.                                                                                                     |
| AUTH-07 | On successful registration, the server <b>MUST</b> return a JWT access token.                                                                                                           |
| AUTH-08 | The client <b>MUST</b> request a nonce via <code>GET /api/auth/challenge/?address=&lt;addr&gt;</code> .                                                                                 |
| AUTH-09 | The server <b>MUST</b> generate a cryptographically random 256-bit nonce and store it with a 5-minute TTL.                                                                              |
| AUTH-10 | The client <b>MUST</b> sign the nonce using EIP-191 <code>personal_sign</code> with the decrypted private key.                                                                          |
| AUTH-11 | The server <b>MUST</b> recover the signer address from the signature and reject if it does not match the claimed address.                                                               |
| AUTH-12 | The nonce <b>MUST</b> be deleted from cache immediately after first verification attempt (single-use).                                                                                  |
| AUTH-13 | On successful login, the server <b>MUST</b> return a JWT with the lowercase wallet address claim and a 7-day lifetime.                                                                  |
| AUTH-14 | The system <b>MUST</b> support MetaMask (EIP-1193 provider) as an alternative login method using the same backend endpoints.                                                            |
| AUTH-15 | When MetaMask is used, VeriFi <b>MUST NOT</b> store or transmit the private key; signing is delegated entirely to MetaMask.                                                             |
| AUTH-16 | JWTs <b>MUST</b> be stored in <code>localStorage["verifi_jwt"]</code> ; wallet address in <code>localStorage["verifi_address"]</code> .                                                 |
| AUTH-17 | All stateful write endpoints <b>MUST</b> validate the JWT and extract the <code>address</code> claim to identify the acting user.                                                       |
| AUTH-18 | Logout <b>MUST</b> clear the JWT, address, and encrypted private key from <code>localStorage</code> .                                                                                   |

### 5.2.2 Hard Claim Creation

Table 5.3: Hard claim creation functional requirements.

| ID     | Requirement                                                                                                                                                        |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLM-01 | A hard claim <b>MUST</b> contain all four fields: <b>asset</b> , <b>direction</b> (Bullish/Bearish), <b>percentage</b> (float, 0–100), <b>until</b> (future date). |
| CLM-02 | The <b>until</b> date <b>MUST</b> be strictly in the future at time of submission; the backend <b>MUST</b> enforce this constraint.                                |
| CLM-03 | The <b>percentage</b> field <b>MUST</b> be between 0 and 100; validated on the backend.                                                                            |
| CLM-04 | A hard claim <b>MUST</b> be linked to the authenticated user’s wallet address as <b>author</b> .                                                                   |
| CLM-05 | The initial status of a new hard claim <b>MUST</b> be <b>undetermined</b> .                                                                                        |
| CLM-06 | The <b>asset</b> field <b>MUST</b> reference a pre-defined <b>Asset</b> record (id, name, symbol, description) from the backend.                                   |
| CLM-07 | The <b>text</b> field contains the user’s free-form natural-language description of their prediction.                                                              |

### 5.2.3 LLM Claim Extraction (Planned)

The endpoint `POST /api/posts/extract-claims/` exists but currently returns `[]` (stub). Full implementation is planned for a future milestone using the Agno framework with a locally hosted language model.

### 5.2.4 Claim Resolution

### 5.2.5 Cryptographic Proofs

The signing architecture is defined; proof download and standalone verification page are planned for Milestone 2.

### 5.2.6 Truth Score

No `truth_score` model field exists yet; this section specifies the intended design for v1.

Table 5.4: LLM claim extraction functional requirements (planned).

| ID      | Requirement                                                                                                                                              |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLM-E01 | The LLM engine <b>MUST</b> extract hard claim candidates from free-form post text.                                                                       |
| CLM-E02 | A valid extracted claim requires all four fields: <code>asset</code> , <code>direction</code> , <code>target_value</code> , <code>timeframe_iso</code> . |
| CLM-E03 | If a required field is missing, the system <b>MUST</b> prompt the user to fill it in explicitly — no inference or generic defaults.                      |
| CLM-E04 | Each extracted claim <b>MUST</b> be presented as a Claim Card for user confirmation, editing, or rejection before posting.                               |
| CLM-E05 | The LLM response schema <b>MUST</b> be: <code>{claims: [{asset, direction, target_value, target_unit, timeframe_iso, language}]}</code> .                |
| CLM-E06 | The LLM engine is stateless; each extraction call is independent.                                                                                        |

Table 5.5: Claim resolution functional requirements.

| ID     | Requirement                                                                                                                                                                    |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RES-01 | A claim <b>MUST</b> be resolved to either <code>confirmed</code> or <code>rejected</code> after its <code>until</code> date passes.                                            |
| RES-02 | Only wallet addresses listed in <code>ADMIN_ADDRESSES</code> (server config) <b>MAY</b> call the claim status update endpoint.                                                 |
| RES-03 | The update endpoint <b>MUST</b> accept <code>{"status": "confirmed" "undetermined" "rejected"}</code> and persist the change.                                                  |
| RES-04 | Automated resolution (scheduled job): a job <b>MUST</b> detect expired claims and query the appropriate oracle to determine outcome. ( <i>Planned — not yet implemented.</i> ) |
| RES-05 | Oracle selection: CoinGecko for crypto assets; Yahoo Finance / Alpha Vantage for traditional assets.                                                                           |
| RES-06 | Oracle failures <b>MUST</b> trigger exponential-backoff retries; after max retries, status remains <code>undetermined</code> .                                                 |
| RES-07 | Resolution results <b>MUST</b> be permanent and auditable; a resolved claim status <b>MUST NOT</b> revert.                                                                     |

Table 5.6: Cryptographic proof functional requirements.

| ID     | Requirement                                                                                                                                                       |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRF-01 | Every hard claim MUST be signed with the author’s secp256k1 private key at creation time.                                                                         |
| PRF-02 | The proof package MUST contain: { <code>claim_payload</code> (JSON), <code>signature</code> (hex), <code>wallet_address</code> , <code>server_timestamp</code> }. |
| PRF-03 | Any authenticated user MUST be able to download a claim’s proof as a local JSON file.                                                                             |
| PRF-04 | A standalone public verification page MUST allow anyone (no login required) to upload a proof JSON and verify its authenticity.                                   |
| PRF-05 | Deleting a post or deactivating an account MUST NOT invalidate a previously generated claim proof.                                                                |
| PRF-06 | Proof verification MUST recover the signer address from the signature and compare it to the stored <code>wallet_address</code> .                                  |

Table 5.7: Truth Score functional requirements.

| ID    | Requirement                                                                                                                                              |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| TS-01 | Each user MUST have a single global numeric Truth Score.                                                                                                 |
| TS-02 | The Truth Score MUST be updated automatically on each claim resolution.                                                                                  |
| TS-03 | The score MUST be weighted by claim difficulty: asset volatility, predicted percentage, and timeframe length. ( <i>Exact formula: TBD — see OQ-01.</i> ) |
| TS-04 | The score normalization algorithm MUST prevent farming via trivially easy predictions.                                                                   |
| TS-05 | The full score history per claim MUST be publicly visible and auditable on the user’s profile.                                                           |
| TS-06 | A claim with status <b>undetermined</b> MUST NOT contribute to the Truth Score until resolved.                                                           |

### 5.2.7 Feed

Table 5.8: Feed functional requirements.

| <b>ID</b> | <b>Requirement</b>                                                                                                                                                              |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FEED-01   | A public feed MUST display all hard claims sorted by creation time (descending).                                                                                                |
| FEED-02   | Each claim card MUST show: author wallet address, asset, direction, percentage target, deadline, and status badge ( <b>Undetermined</b> / <b>Confirmed</b> / <b>Rejected</b> ). |
| FEED-03   | The feed MUST be filterable by asset and asset category without page reload.                                                                                                    |
| FEED-04   | Feed data MUST be accessible to unauthenticated users.                                                                                                                          |
| FEED-05   | The feed MUST load in under 2 seconds under normal conditions.                                                                                                                  |

Table 5.9: User profile functional requirements.

| ID      | Requirement                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------------------|
| PRF-P01 | Every wallet address MUST have a publicly accessible profile page.                                                               |
| PRF-P02 | A profile MUST display: wallet address, Truth Score, and full claim history.                                                     |
| PRF-P03 | Claim history MUST include a per-claim proof download link.                                                                      |
| PRF-P04 | Users MUST be able to filter another user's profile by wallet address (search).                                                  |
| PRF-P05 | The profile page MUST allow the authenticated owner to reveal their encrypted private key temporarily (60-second auto-hide).     |
| PRF-P06 | The revealed private key MUST be displayed partially obfuscated ( <code>first8...last8</code> ) with a copy-to-clipboard option. |

### 5.2.8 User Profiles

## 5.3 Non-Functional Requirements

### 5.3.1 Security

### 5.3.2 Privacy and GDPR / KVKK

### 5.3.3 Performance and Reliability

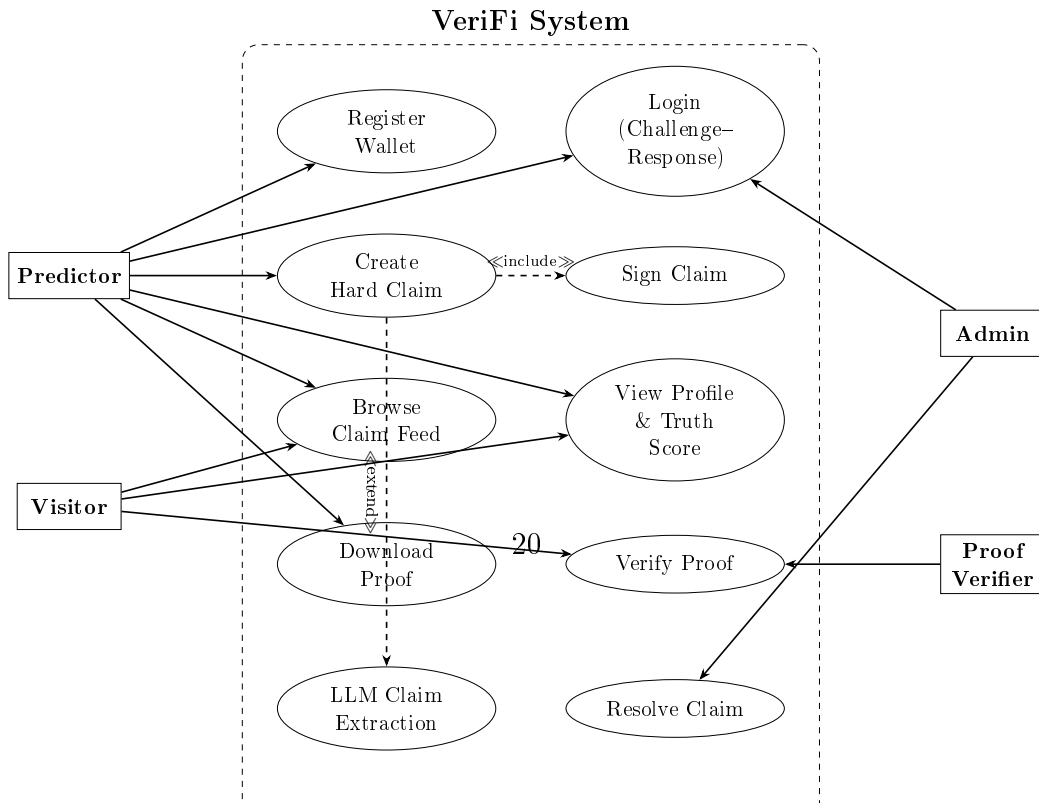


Table 5.10: Security non-functional requirements.

| Requirement             | Detail                                                                                                                      |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Private key isolation   | Private key MUST never be transmitted to any server. Only the derived public address is known to the server.                |
| Local key encryption    | Private key MUST be encrypted with AES-256-GCM + PBKDF2-SHA256 (100,000 iterations) before storage.                         |
| Nonce replay prevention | Nonces are single-use; deleted from cache immediately after first verification attempt. TTL: 5 minutes.                     |
| JWT security            | JWT MUST be signed with <code>SECRET_KEY</code> (environment variable in production). Lifetime: 7 days.                     |
| CORS                    | <code>CORS_ALLOWED_ORIGINS</code> MUST be restricted to the known frontend origin; wildcard MUST NOT be used in production. |
| Admin authorization     | Only addresses in <code>ADMIN_ADDRESSES</code> may call claim resolution endpoints; checked server-side on every request.   |

Table 5.11: Privacy and data protection non-functional requirements.

| Requirement              | Detail                                                                                               |
|--------------------------|------------------------------------------------------------------------------------------------------|
| No PII stored            | No email, phone number, real name, or other personally identifiable information is stored.           |
| Post text deletion       | Users MUST be able to request deletion of post text (GDPR / KVKK right to erasure).                  |
| Claim payload permanence | Signed claim payloads are mathematical accountability data and MAY persist after post text deletion. |
| Pseudonymous identity    | Wallet address is a pseudonymous identifier; users choose how much real-world identity to attach.    |



Table 5.12: Performance and reliability non-functional requirements.

| <b>Requirement</b>    | <b>SLA / Detail</b>                                                      |
|-----------------------|--------------------------------------------------------------------------|
| Feed initial load     | < 2 seconds under normal conditions                                      |
| LLM claim extraction  | < 10 seconds per extraction call (when implemented)                      |
| Claim resolution job  | Must run within 5 minutes of <code>until</code> date expiry              |
| Proof verification    | < 1 second (client-side computation)                                     |
| Oracle retries        | Exponential backoff on oracle failure; resolution job MUST be idempotent |
| Claim data durability | Claim records MUST survive post deletions and account deactivations      |

# Chapter 6

## DESIGN

### 6.1 Information Structure

The core data models and their relationships are described below.

Table 6.1: `WalletUser` — registered user identity.

| Field                   | Type                       | Constraints                                            |
|-------------------------|----------------------------|--------------------------------------------------------|
| <code>id</code>         | Auto PK                    | —                                                      |
| <code>address</code>    | <code>CharField(42)</code> | Unique, lowercase Ethereum address (0x + 40 hex chars) |
| <code>created_at</code> | <code>DateTimeField</code> | Auto-set on creation                                   |

Table 6.2: `Asset` — tradable financial asset.

| Field                    | Type                        | Constraints    |
|--------------------------|-----------------------------|----------------|
| <code>id</code>          | Auto PK                     | —              |
| <code>name</code>        | <code>CharField(100)</code> | e.g. “Bitcoin” |
| <code>symbol</code>      | <code>CharField(10)</code>  | e.g. “BTC”     |
| <code>description</code> | <code>TextField</code>      | Optional       |

Table 6.3: `HardClaim` — the canonical verifiable financial prediction entity.

| Field                   | Type                         | Constraints                                                                          |
|-------------------------|------------------------------|--------------------------------------------------------------------------------------|
| <code>id</code>         | Auto PK                      | —                                                                                    |
| <code>author</code>     | FK → <code>WalletUser</code> | Nullable (system/anonymous claims)                                                   |
| <code>text</code>       | <code>TextField</code>       | User's free-form prediction description                                              |
| <code>asset</code>      | FK → <code>Asset</code>      | Required                                                                             |
| <code>direction</code>  | <code>CharField(20)</code>   | "Bullish" or "Bearish"                                                               |
| <code>percentage</code> | <code>FloatField</code>      | 0–100; predicted price move magnitude                                                |
| <code>until</code>      | <code>DateTimeField</code>   | Must be > <code>created_at</code> (DB constraint)                                    |
| <code>created_at</code> | <code>DateTimeField</code>   | Auto-set                                                                             |
| <code>status</code>     | <code>CharField(12)</code>   | <code>undetermined</code> (default) · <code>confirmed</code> · <code>rejected</code> |

Table 6.4: `Post` (legacy) — free-form social post; will be deprecated in a future milestone.

| Field                   | Type                         | Constraints                  |
|-------------------------|------------------------------|------------------------------|
| <code>id</code>         | Auto PK                      | —                            |
| <code>author</code>     | FK → <code>WalletUser</code> | CASCADE delete               |
| <code>content</code>    | <code>TextField(500)</code>  | Max 500 characters           |
| <code>created_at</code> | <code>DateTimeField</code>   | Auto-set; ordered descending |

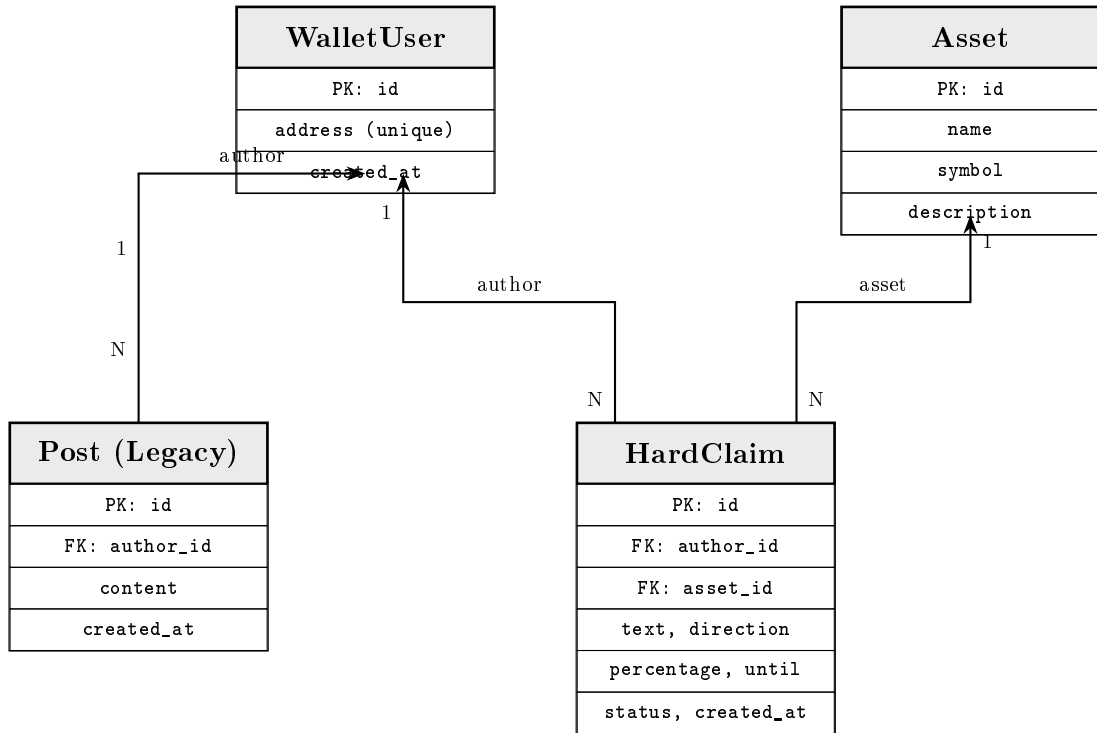


Figure 6.1: Entity-Relationship diagram of the core data models. `HardClaim` references both `WalletUser` (author) and `Asset`. `Post` (legacy) also references `WalletUser`.

## 6.2 Information Flow

**Authentication flow (native wallet):** The browser generates a mnemonic and derives a `secp256k1` key pair entirely client-side. The address is registered with the backend via `POST /api/auth/register/`. On login, the browser requests a nonce from `GET /api/auth/challenge/`, signs it with the private key (EIP-191), and sends the signature to `POST /api/auth/login/`. The backend uses `eth_account.recover_message()` to recover the signing address and issues a JWT on match.

**Claim creation flow:** An authenticated user submits the hard claim form. The frontend sends `POST /api/posts/hard-claims/` with a Bearer JWT header. The backend validates the JWT, extracts the wallet address as the claim author, validates all claim fields server-side, and stores the record with status `undetermined`.

**Claim resolution flow:** After a claim's `until` date passes, an admin (or, in future milestones, an automated oracle job) submits `PATCH /api/`

posts/hard-claims/{id}/update-status/. The backend verifies admin identity, updates status, and triggers Truth Score recalculation for the claim author.

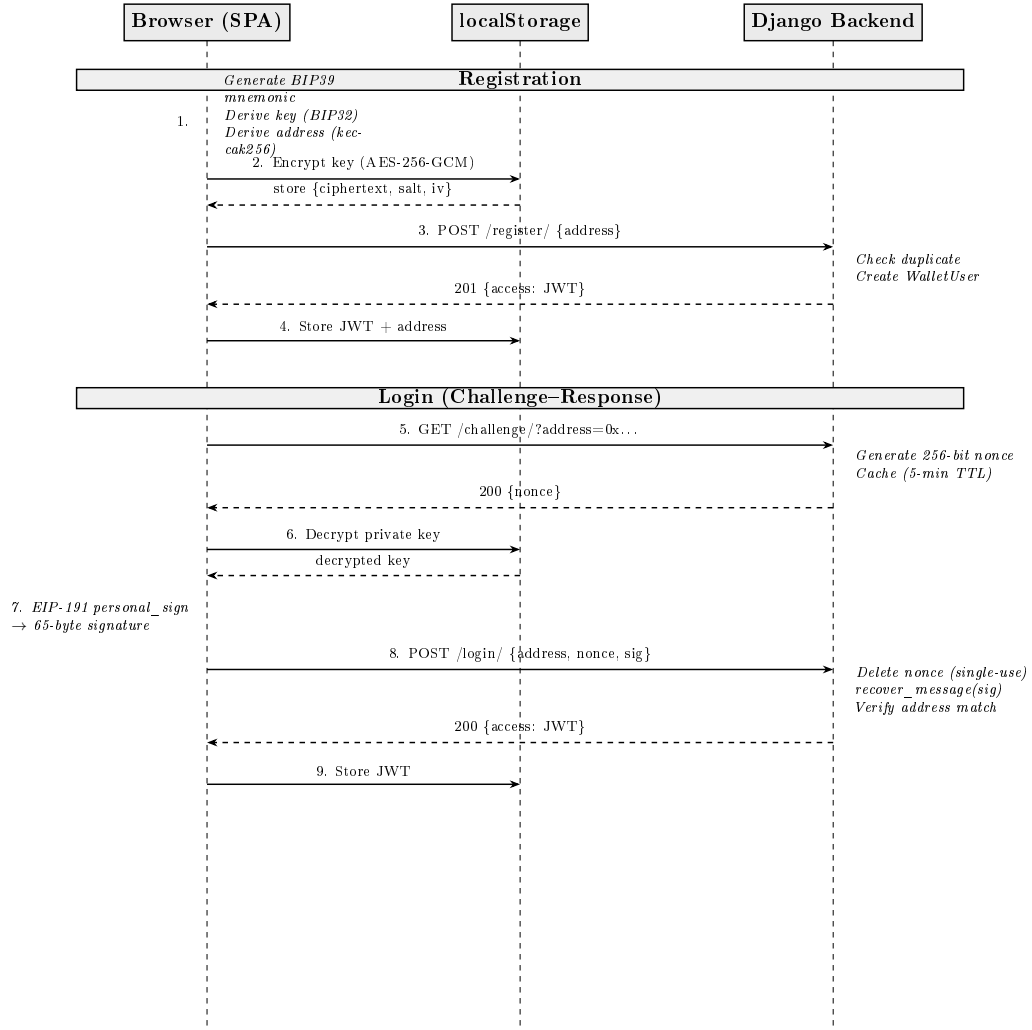


Figure 6.2: Sequence diagram for native wallet registration and challenge-response login.

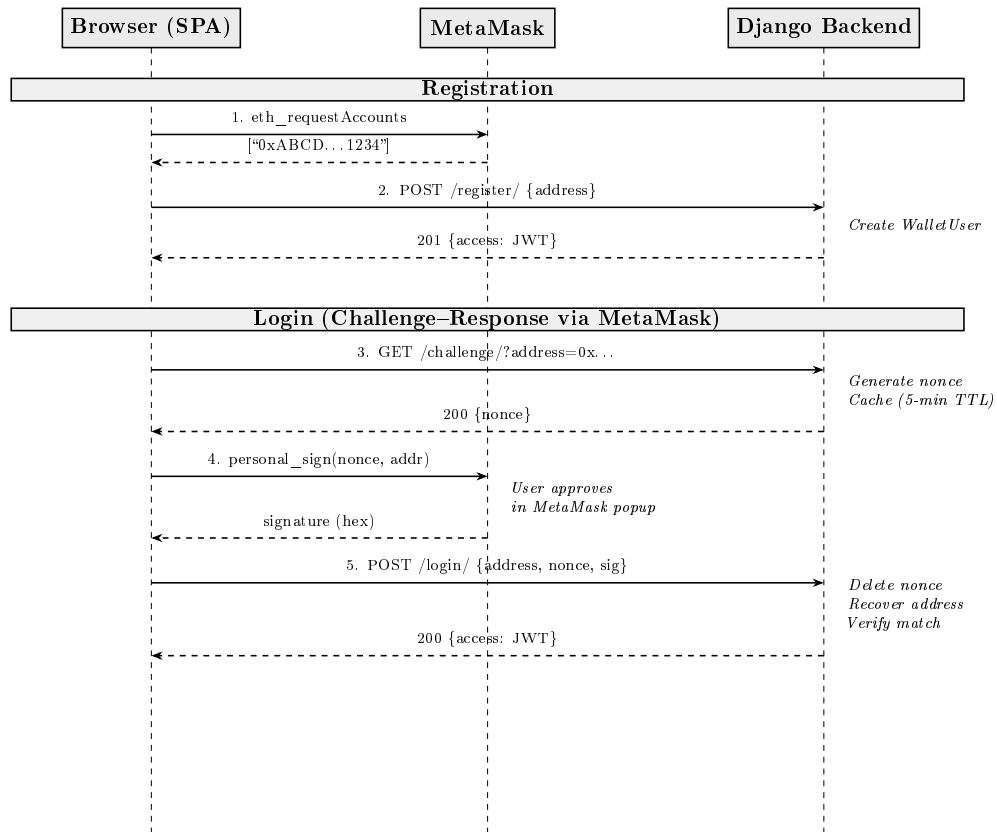


Figure 6.3: Sequence diagram for MetaMask registration and login. The private key never leaves the MetaMask vault.

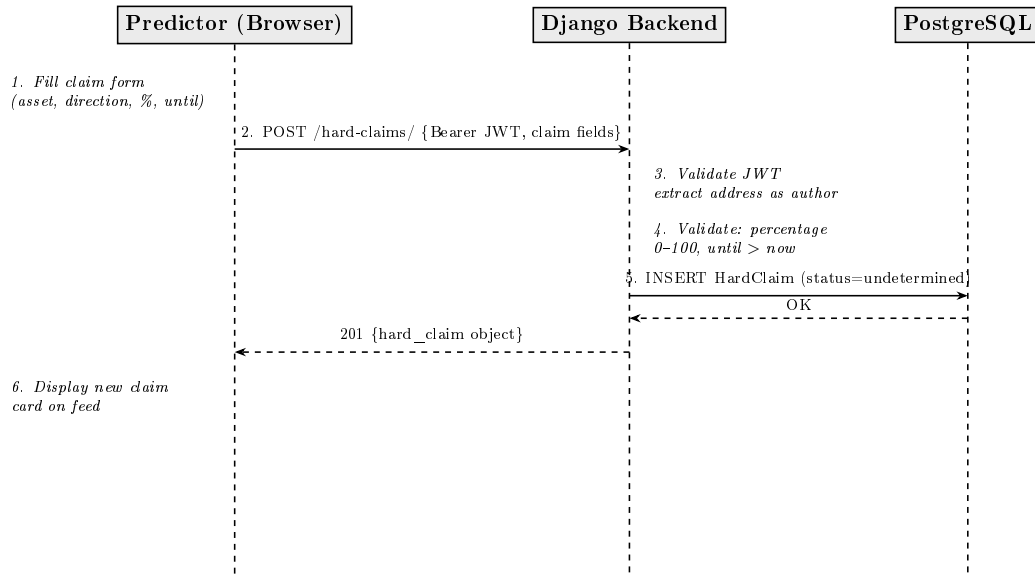


Figure 6.4: Sequence diagram for hard claim creation. The backend validates all fields and stores the claim with initial status **undetermined**.

## 6.3 System Design

The system follows a strict client-server separation. The frontend is a static SPA deployed on Vercel that communicates with the backend exclusively through the REST API over HTTPS. The backend is stateless: all authenticated state is in the JWT token carried by the client. The private key never leaves the browser.

The REST API is organized into two Django apps:

- **accounts** — authentication endpoints: `/api/auth/register/`,  
`/api/auth/challenge/`, `/api/auth/login/`
- **posts** — content endpoints: `/api/posts/`,  
`/api/posts/hard-claims/`, `/api/posts/assets/`,  
`/api/posts/extract-claims/`

Cross-origin requests from the frontend origin (Vercel domain and `localhost:5173` in development) are handled by `django-cors-headers`.

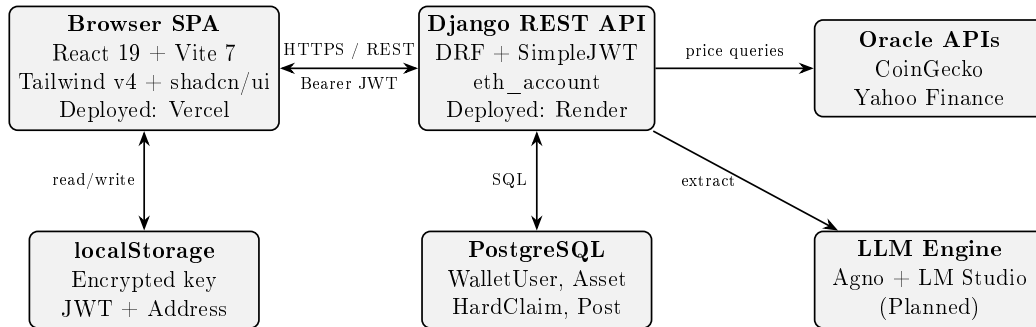


Figure 6.5: High-level system architecture. The frontend communicates with the stateless backend over HTTPS; the private key never leaves the browser.

## 6.4 Authentication Design

The authentication system is designed around the invariant that the private key is never transmitted to or stored by the server. The server only sees a wallet address (a 20-byte Keccak-256 hash of a public key) and a cryptographic signature over a server-issued nonce.

The native VeriFi wallet uses BIP39 (12-word mnemonic, 128 bits of entropy), BIP32/BIP44 key derivation (path `m/44'/60'/0'/0/0`), and standard Ethereum address derivation. The private key is encrypted with AES-256-GCM using a key derived from the user's passphrase via PBKDF2-SHA256 (100,000 iterations) and stored as a JSON ciphertext object in browser `localStorage`.

Login uses EIP-191 `personal_sign`: the server generates a random hex nonce (32 bytes, 5-minute TTL, single-use), the client signs it, and the server uses `eth_account.recover_message()` to verify the signature. The following table summarises the security properties of both wallet modes.

Figure 6.6 presents the combined authentication flowchart, showing how the system routes a user through either the native wallet or MetaMask path before converging on the shared challenge–response verification.



Table 6.5: Security properties of native wallet vs. MetaMask authentication.

| Property                 | Native Wallet                        | MetaMask                          |
|--------------------------|--------------------------------------|-----------------------------------|
| Private key location     | localStorage (AES-256-GCM encrypted) | MetaMask encrypted vault          |
| Private key transmitted? | Never                                | Never                             |
| Replay attack protection | Single-use nonce (5-min TTL)         | Single-use nonce (5-min TTL)      |
| Signature scheme         | EIP-191 personal_sign (secp256k1)    | EIP-191 personal_sign (secp256k1) |
| Session mechanism        | Stateless JWT (7-day expiry)         | Stateless JWT (7-day expiry)      |

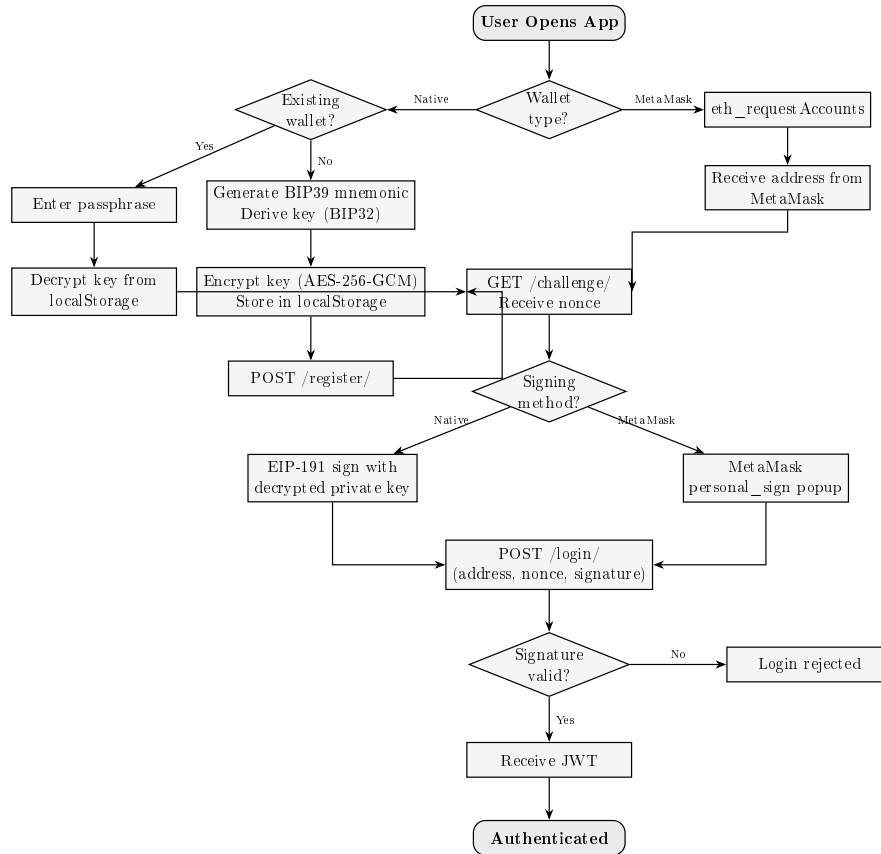


Figure 6.6: Combined authentication flowchart showing native wallet and MetaMask login paths converging at the shared EIP-191 challenge-response verification.

## 6.5 User Interface Design

The frontend implements the following main views:

- **Login / Registration Page:** Wallet generation wizard, mnemonic display, passphrase setup, and MetaMask connect button.
- **Feed Page:** Public list of hard claims rendered as PostCards, each showing author address, asset, direction, percentage, deadline, and status badge. New Claim dialog accessible to authenticated users.
- **Post Detail Page:** Expanded view of a single post with all associated hard claims and proof download options.
- **Profile Page:** User's wallet address (with 60-second timed private key reveal, partially obfuscated), Truth Score, and complete claim history.
- **Proof Verification Page:** Standalone page (no login required) for uploading and verifying a downloaded proof JSON file.
- **Post Creation Page:** Free-form text composer with optional LLM-assisted claim extraction.
- **Claim Review Page:** Displays extracted Claim Cards for user confirmation before publishing.

The design language uses Tailwind CSS v4 utility classes and the shadcn/ui component library for consistent, accessible UI components (buttons, dialogs, badges, alerts, selects, inputs, textareas).

*Note: UI wireframes, mockups, and screenshots of the implemented interface will be included in the final report.*

## 6.6 Design Decisions and Constraints

Table 6.6: Key design decisions and their rationale.

| Decision              | Choice                                            | Rationale                                                                     |
|-----------------------|---------------------------------------------------|-------------------------------------------------------------------------------|
| Claim target format   | <code>percentage</code> (0–100 float)             | More universal than absolute price; works across asset classes and currencies |
| Claim types (v1)      | Hard claims only                                  | Soft claims require a mature voting system; deferred to v2                    |
| Claim resolution (v1) | Admin-controlled                                  | Automated oracle resolution is planned but not yet implemented                |
| Timestamping          | Backend server timestamp                          | Sufficient for v1; blockchain anchoring is future work                        |
| Reputation            | Single global Truth Score                         | Domain-specific scores deferred to v2                                         |
| Identity              | Wallet address only                               | Privacy-preserving; no PII stored                                             |
| GDPR handling         | Post text deletable; claim payload permanent      | Claim data is accountability data, not personal commentary                    |
| Authentication        | Passwordless wallet signature                     | No password storage; cryptographic proof of key ownership                     |
| Key storage           | <code>localStorage</code> (AES-256-GCM encrypted) | Acceptable for v1 target scale; hardware wallet support is future work        |

# Chapter 7

## IMPLEMENTATION AND TESTING

### 7.1 Implementation

As of the midterm (March 2026), the following features are implemented and functional.

**Authentication system:**

- Native wallet: BIP39 mnemonic generation, secp256k1 key derivation, passphrase-based AES-256-GCM encryption, `localStorage` storage, challenge-response JWT login.
- MetaMask: EIP-1193 `eth_requestAccounts` and `personal_sign` integration using the same backend challenge-response flow.
- Unified login UI supporting both wallet modes on a single page.
- JWT-based session management (7-day token lifetime, stateless).
- Logout: clears JWT, wallet address, and encrypted key from `localStorage`.

**Hard claims:**

- Hard claim creation form with asset dropdown, direction selector, percentage input, and date picker.
- Backend validation: percentage range (0–100), future deadline enforcement (DB-level constraint), authenticated author linking.
- Public hard claim feed rendered as PostCard components.

- Hard claim filtering by author address (`?address=` query parameter).
- Admin-only manual claim status update endpoint.
- Asset list API with pre-seeded assets (Bitcoin, Ethereum, US Dollar, and others).

#### **User interface:**

- Feed page with PostCard components showing all hard claim fields and status badges.
- Profile page with timed private key reveal (60-second auto-hide, partial obfuscation, copy-to-clipboard).
- Responsive layout with navbar, Tailwind v4 theming, and shadcn/ui components.

#### **Pending for upcoming milestones:**

- Cryptographic proof signing, proof download UI, and standalone proof verification page (Milestone 2).
- Truth Score model, calculation formula, and public display (Milestone 2/3).
- Automated oracle-based claim resolution with exponential-backoff retries (Milestone 3).
- LLM claim extraction (currently a stub returning `[]`); full implementation in progress.
- Feed filtering by asset (Milestone 3).
- Post text deletion for GDPR / KVKK compliance.

## **7.2 AI-Integrated Claim Extraction Engine**

As part of the VeriFi project, a specialized AI engine was developed to transform unstructured financial text into verifiable “Hard Claims.” This section details the framework selection, iterative development, and benchmarking processes conducted as of March 2026.

### 7.2.1 Framework Research and Selection

A comprehensive evaluation of Large Language Model (LLM) orchestration frameworks was performed to identify the most suitable architecture for niche financial claim extraction. The following frameworks were researched:

- **MLX & Ollama:** Evaluated for local inference capabilities.
- **LlamaIndex & Langchain:** Analyzed for their broad RAG (Retrieval-Augmented Generation) capabilities.
- **Agno (formerly Phidata):** Selected as the primary framework due to its lightweight agentic architecture. Agno’s superior support for structured Pydantic outputs and its specialized focus on agentic workflows aligned perfectly with the project’s requirement for high-precision data extraction.

### 7.2.2 Implementation and Prompt Engineering

The development of the extraction agent followed an evolutionary approach:

- (i) **Initial Phase:** A baseline extractor was implemented using standard completion calls to identify simple financial entities.
- (ii) **Advanced System Prompting:** The system prompt was iteratively refined into a robust instruction set. This version incorporates few-shot prompting and strict schema enforcement to ensure that extracted claims include all necessary metadata (asset, direction, percentage, and deadline) while minimizing hallucinations in noisy text.

### 7.2.3 Model Benchmarking and Evaluation

A multi-stage testing protocol was established to determine the optimal model for production.

**Phase 1: Exploratory Testing (20-Sample Base Instruction).** The following models were tested using a curated set of 20 complex financial sentences: *zai-org/glm-4.6v-flash*, *mistralai/ministral-8b*, *deepseek/deepseek-r1-0528-qwen*, *liquid/lfm2.5-1.2b*, *ministral-3-8b-instruct*, *allenai/olmocr-2-7b*, *qwen2.5-7b-instruct-1m*, *hermes-3-llama-3.1-8b*, *gemma-3-27b-it*, *qwen/qwen3-vl-8b*, *qwen/qwen3.5-9b*, *meta-llama-3-8b-instruct*, and *google/gemma-3-4b*.

During this phase, **hermes-3-llama-3.1-8b** exhibited the highest instruction-following capability and structural consistency.

**Phase 2: Synthetic Scale Testing.** A larger dataset consisting of 1,000 AI-generated samples was used to stress-test the extraction logic. In this scaled environment, **ministral-3-8b-instruct** emerged as the most efficient model, balancing high extraction accuracy with optimized token usage.

#### 7.2.4 Current Work: The Golden Dataset

To ensure the system’s reliability against professional-grade financial discourse, work has commenced on a “Golden Dataset.” This dataset moves beyond synthetic data and is curated from high-authority real-world sources, including:

- Corporate annual reports and bank summary statements.
- Central Bank publications and monetary policy reports.
- Financial analyses from institutions such as **Goldman Sachs**.
- Opinions and forecasts from prominent economists and financial columnists.

The objective of this dataset is to fine-tune the extraction agent’s ability to isolate “hard claims” from the nuanced and often hedged language of professional financial reporting.

### 7.3 Testing

**Security testing.** An XSS vulnerability scan was conducted on 2026-03-17 using XSSStrike v3.1.5 targeting both the frontend (`localhost:5173`) and backend (`localhost:8000`) at commit `861894b`. All 28 fuzzer payloads — including `<script>`, `<svg>`, and event handler injections (`onload`, `onclick`) — were blocked. No XSS vulnerabilities were detected. React’s automatic content escaping (no `dangerouslySetInnerHTML` usage) and Django REST Framework’s JSON-only response format both contribute to this result.

**Manual testing.** Authentication flows (native wallet and MetaMask), hard claim creation, and feed display have been tested manually in the development environment. The challenge-response authentication flow has been verified end-to-end: nonce issuance, EIP-191 signing in the browser, server-side `eth_account` signature recovery, and JWT issuance.

**LLM benchmark.** A benchmark dataset has been designed to evaluate LLM claim extraction quality. The dataset contains examples in English and Turkish across four categories: (1) single-claim factual predictions, (2)

multi-claim sentences, (3) combined quantitative predictions, and (4) noise / near-miss inputs that should not produce claims (e.g., intent statements, non-quantified commentary). Results will be reported in the final submission.

Automated unit and integration testing with a structured test suite are planned for the upcoming milestones.

## 7.4 Deployment

Deployment to production is planned and will be completed before the final report submission. The intended deployment architecture is as follows:

- **Frontend:** Vercel, building from the `frontend/` directory with `pnpm` build. Configuration in `frontend/vercel.json`. SPA routing will be handled via `rewrites` config.
- **Backend:** Render, running `gunicorn` as the WSGI server. Configuration in `backend/render.yaml`. Environment variables (`SECRET_KEY`, `DATABASE_URL`, `ADMIN_ADDRESSES`, `CORS` origins) will be managed through Render's dashboard.
- **Database:** SQLite in development; PostgreSQL managed by Render in production.

A deployment diagram and full build/run instructions will be included in the final report.



# Chapter 8

## RESULTS

As of the midterm submission, the authentication system and core hard claim pipeline are fully operational. Users can register and log in with both a native VeriFi wallet and MetaMask, create hard claims through a structured form, and browse the public claim feed. The admin claim resolution mechanism is functional, enabling manual lifecycle management from **undetermined** to **confirmed** or **rejected**. The platform is live in production on Vercel (frontend) and Render (backend).

Milestone 1 (March 8, 2026) was completed on schedule, delivering the full authentication system. The hard claim data model, feed, and basic profile pages were delivered as part of subsequent sprint work ahead of the midterm.

The LLM claim extraction feature is partially implemented. The extraction endpoint exists and is reachable; the LLM benchmark dataset has been designed and populated with English and Turkish examples. Full integration with the Agno framework and a hosted language model is in progress and targeted for Milestone 2.

The cryptographic proof system (signing at claim creation, proof download, and standalone verification) and the Truth Score calculation are the two most significant remaining features of the platform and are targeted for Milestones 2 and 3 respectively.

## Chapter 9

# CONCLUSION

VeriFi aims to bring cryptographic accountability to financial social media — a domain currently dominated by unverifiable claims and selective recall. By anchoring every prediction to a signed proof tied to a cryptographic wallet identity, the platform makes financial predictions permanently attributable and verifiable, even after the original post is deleted.

At the midterm stage, the core infrastructure is in place: a full wallet-based authentication system supporting both native BIP39 wallets and MetaMask, the hard claim creation and feed pipeline, manual admin resolution, and a live production deployment. The authentication flow represents a significant technical achievement, implementing the full BIP39/BIP32/secp256k1/EIP-191 stack client-side with AES-256-GCM key protection, while maintaining full compatibility with the Ethereum ecosystem.

The remaining work focuses on completing the reputation and proof systems: cryptographic proof signing and download, the standalone verifier page, oracle-based automated resolution, and the Truth Score formula and calculation engine. These features constitute the distinctive value proposition of VeriFi and will be the primary development focus for the second half of the semester.

# Bibliography

- [1] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ECDSA). *International Journal of Information Security*, 1(1):36–63, 2001.
- [2] Audun Jøsang, Roslan Ismail, and Colin Boyd. A survey of trust and reputation systems for online service provision. *Decision Support Systems*, 43(2):618–644, 2007.
- [3] Marek Palatinus, Pavol Rusnak, Aaron Voisine, and Sean Bowe. BIP-39: Mnemonic code for generating deterministic keys. Bitcoin Improvement Proposals, 2013. Available at <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>.
- [4] Jack Peterson, Joseph Krug, Micah Zoltu, Austin Williams, and Stephanie Alexander. Augur: a decentralized oracle and prediction market platform. Technical report, Forecast Foundation, 2019. Version 2.0. Available at <https://github.com/AugurProject/whitepaper>.
- [5] Geth Team. EIP-191: Signed data standard. Ethereum Improvement Proposals, 2016. Available at <https://eips.ethereum.org/EIPS/eip-191>.
- [6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [7] Justin Wolfers and Eric Zitzewitz. Prediction markets. *Journal of Economic Perspectives*, 18(2):107–126, 2004.
- [8] Pieter Wuille. BIP-32: Hierarchical deterministic wallets. Bitcoin Improvement Proposals, 2012. Available at <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>.